

ESTRATEGIA DE PRUEBAS — ONBOARDING DIGITAL

Rol: Analista de QA

Objetivo: Definir una estrategia de pruebas que permita identificar y prevenir los riesgos más importantes del proceso de onboarding digital, priorizando aquellos que puedan generar impactos transaccionales, inconsistencias de datos o interrupciones del servicio.

1. Priorización de pruebas y automatización

Durante el primer mes se priorizarán los escenarios de mayor riesgo para el negocio:

1. **Idempotencia del pago:** evitar cobros duplicados ante callbacks repetidos.
2. **Flujo E2E:** validar el proceso completo desde el login hasta la confirmación del pago.
3. **Tracking:** verificar su correcta creación y consulta.
4. **Procesos asíncronos:** validar el procesamiento de colas y actualización de estados.
5. **Casos negativos de API:** comprobar el manejo de productos inexistentes y payloads inválidos.

La prioridad se establece considerando principalmente el **impacto en el negocio, riesgo transaccional y frecuencia de posibles fallos**.

2. Distribución de las pruebas por nivel

La estrategia contempla diferentes niveles de prueba, asignando cada escenario al nivel donde pueda ser validado de manera más eficiente.

Pruebas unitarias

Las pruebas unitarias estarán enfocadas en la lógica interna de cálculo, principalmente precios, impuestos y descuentos implementados en las Lambdas.

Este tipo de prueba permite validar lógica pura sin depender de servicios externos, bases de datos o componentes de infraestructura.

Pruebas de integración y API

Las pruebas de integración y API estarán orientadas a validar los contratos entre los diferentes servicios.

Se incluirán escenarios relacionados con:

- Creación y consulta de tracking.
- Carrito de compras.
- Procesamiento de pagos.
- Idempotencia.
- Procesamiento de mensajes y colas.

- Validación de entradas y respuestas de los servicios.

Pruebas End-to-End

Las pruebas E2E validarán el comportamiento completo del portal desde la perspectiva del usuario.

El objetivo será comprobar que las diferentes transiciones del proceso sean correctas, por ejemplo:

Login → Selección → Carrito → Pago → Confirmación

De esta manera no solo se valida cada componente individualmente, sino también la integración de toda la cadena funcional.

Pruebas de carga

Las pruebas de rendimiento estarán enfocadas principalmente en los endpoints críticos, considerando como referencia el volumen pico establecido para el sistema.

El objetivo será determinar cómo responde la aplicación bajo carga y detectar posibles problemas de latencia, errores, throttling o saturación de recursos.

3. Pruebas que no se automatizarán inicialmente

No todos los escenarios deben automatizarse inmediatamente. Una buena estrategia de QA también debe considerar el costo de mantenimiento y la estabilidad de las dependencias.

Durante los primeros seis meses se propone no automatizar directamente la **validación de identidad y firma digital**, debido a que los proveedores externos no disponen de un sandbox suficientemente estable. En estos casos se utilizarán pruebas de contrato mediante Pact y mocks controlados.

Tampoco se priorizará la automatización del **backoffice de parametrización**, debido a su baja frecuencia de cambios y al costo que tendría mantener los datos necesarios para estas pruebas.

Las **pruebas exploratorias de experiencia de usuario** continuarán realizándose manualmente, debido a que requieren observación, criterio y capacidad de detectar comportamientos que no necesariamente están definidos previamente.

Finalmente, la **geolocalización** será validada manualmente utilizando datos de prueba geofenciados, debido a la dependencia de coordenadas y condiciones reales.

Esto no significa que estos escenarios nunca puedan automatizarse, sino que se priorizarán posteriormente cuando el costo y el beneficio de automatizarlos sean favorables.

4. Gestión de datos y ambientes

Para garantizar que las pruebas sean repetibles y no dependan de información generada por ejecuciones anteriores, cada prueba deberá trabajar con un estado controlado.

Se utilizará una fixture automática que ejecute un proceso de limpieza mediante el endpoint /reset antes y después de cada prueba.

Los datos serán completamente sintéticos y se generarán dinámicamente utilizando uuid4. De esta manera se evita que diferentes pruebas compartan información y se reducen las posibilidades de colisiones.

Los mocks correspondientes al servicio de pago y a las colas podrán levantarse una sola vez por sesión, mientras que el estado utilizado por cada prueba será aislado individualmente.

En ambientes compartidos como staging se utilizarán prefijos para los identificadores de tracking, por ejemplo:

- QA-
- DEV-

Esto permitirá identificar el origen de los datos y reducir las colisiones entre equipos sin necesidad de crear una base de datos independiente para cada grupo.

5. Manejo de procesos asíncronos

Uno de los puntos importantes de esta estrategia es evitar esperas fijas mediante `time.sleep()`.

En procesos asíncronos, las pruebas deben esperar realmente a que el sistema alcance el estado esperado.

Para esto se utilizará un mecanismo de polling similar al siguiente:

```
poll_until_state(message_id, "COMPLETED", timeout=10s)
```

El mecanismo utilizará un backoff exponencial para reducir la cantidad de consultas innecesarias:

0.1 segundos → 0.2 segundos → 0.4 segundos → ... hasta un máximo de 2 segundos.

Si el sistema devuelve un estado FAILED cuando se esperaba COMPLETED, la prueba deberá fallar inmediatamente.

Si se alcanza el tiempo máximo de espera sin llegar al estado esperado, la prueba también deberá fallar.

Esto evita falsos positivos y permite identificar claramente cuándo un proceso asíncrono no está funcionando correctamente.

Para validar reintentos y eventos duplicados, la cola deberá permitir consultar los mensajes procesados. La prueba verificará que exista un único registro para cada `idempotency_key`, independientemente de cuántas veces se haya publicado el mismo evento.

6. Integración con servicios de terceros

Los servicios externos representan una dependencia importante para las pruebas, por lo que se propone utilizar diferentes estrategias dependiendo de su disponibilidad.

Cuando el proveedor disponga de un ambiente estable, se utilizarán **pruebas de contrato mediante Pact** para validar que la comunicación entre sistemas respete los contratos definidos.

Cuando el proveedor presente problemas de disponibilidad o lentitud, se utilizarán mocks locales capaces de simular diferentes niveles de latencia.

Cuando el servicio externo se encuentre completamente fuera de servicio, se utilizarán stubs con respuestas controladas para validar que nuestra aplicación maneje correctamente los errores.

La regla principal será que **ninguna prueba ejecutada en el pipeline principal de CI dependa de un servicio externo real**.

Las pruebas de contrato contra proveedores externos podrán ejecutarse durante procesos programados, por ejemplo, en una ejecución nocturna.

7. Estrategia de rendimiento

El escenario prioritario para rendimiento será la **generación y validación del enlace de pago**, debido a que corresponde a una de las operaciones más críticas del proceso y puede presentar una mayor latencia.

Como línea inicial de aceptación se proponen los siguientes valores:

- P95 de latencia general: menor a 500 ms.
- P99 de latencia general: menor a 1.000 ms.
- P95 del endpoint de pago: menor a 400 ms.
- Tasa de errores HTTP: menor al 1 %.
- Violaciones de idempotencia: 0.
- Procesos asíncronos completados dentro de 10 segundos: 100 %.

Sin embargo, estos valores deben considerarse como objetivos iniciales y no como valores definitivos.

Antes de establecer una línea base de rendimiento, se deberán comparar los resultados con métricas reales obtenidas de CloudWatch, incluyendo:

- P95 de API Gateway.
- Duración de las Lambdas.
- Throttling de DynamoDB.
- Capacidad y comportamiento de SQS.
- Concurrencia de Lambda.
- Conexiones utilizadas por DynamoDB.

Los resultados obtenidos en staging no deberán extrapolarse directamente a producción, debido a que existen diferencias de infraestructura, concurrencia y volumen.

Para identificar el componente responsable de una degradación se utilizará el `tracking_id` como elemento de correlación a través de:

X-Ray → API Gateway → Lambda → DynamoDB / SQS

Esto permitirá realizar un análisis más preciso de dónde se encuentra el cuello de botella.

8. Integración con CI/CD

La automatización se integrará progresivamente al pipeline de CI/CD para proporcionar retroalimentación rápida al equipo de desarrollo.

En cada commit se ejecutarán validaciones rápidas como:

- Lint mediante Ruff.
- Escaneo de seguridad mediante Bandit.
- Detección de credenciales expuestas.

En cada Pull Request o merge se ejecutarán las pruebas principales:

- Pruebas de API.
- Pruebas de idempotencia.
- Pruebas de procesos asíncronos.
- Pruebas E2E.
- Pruebas de carga según la estrategia definida.

Durante la ejecución nocturna se ejecutarán las pruebas que requieran mayor tiempo o dependencias externas, incluyendo:

- Pruebas de contrato contra sandboxes externos.
- Suite completa.
- Ejecuciones con datos frescos.

Cuando una prueba crítica falle y deba bloquear un despliegue, el pipeline deberá generar evidencia suficiente para facilitar el análisis.

El reporte HTML será publicado como artefacto del pipeline y el equipo recibirá una notificación con el enlace correspondiente.

La prueba de idempotencia será considerada un criterio de bloqueo, debido al impacto que podría generar un procesamiento duplicado de pagos.

No se aprobará el pase a producción mientras los criterios mínimos definidos para el despliegue no se encuentren correctamente validados.

9. Métricas de calidad a seis meses

Después de implementar la estrategia, será importante medir si las pruebas realmente están generando valor para el equipo y para el negocio.

Se propone realizar seguimiento a cinco métricas principales.

Defectos escapados a producción

Permite conocer cuántos defectos no fueron detectados durante las etapas de prueba y llegaron al ambiente productivo.

Tiempo medio de detección de regresiones

Mide cuánto tiempo tarda el proceso de QA en identificar que una modificación introdujo un comportamiento incorrecto.

Porcentaje de despliegues bloqueados automáticamente

Permite demostrar que el pipeline no solo ejecuta pruebas, sino que también funciona como una barrera de seguridad antes de producción.

Cobertura de riesgos críticos

Permite conocer qué porcentaje de los riesgos identificados en la matriz de riesgos cuenta con pruebas automatizadas.

Tiempo de ejecución del pipeline

Es importante controlar que el crecimiento de la automatización no genere pipelines excesivamente lentos que terminen afectando el ciclo de desarrollo.

10. Matriz de trazabilidad

Finalmente, se utilizará una matriz de trazabilidad para relacionar cada riesgo con su prueba automatizada, evidencia y criterio de bloqueo.

El riesgo relacionado con el **doblo cobro por callback repetido** será validado mediante la prueba `test_counter_refleja_un_solo_cobro_logico`. La evidencia estará disponible en el reporte transaccional y el criterio esperado será que el contador refleje un único cobro lógico.

Para el riesgo relacionado con un **tracking no generado o no consultable**, se ejecutará `test_crear_tracking_retorna_id_unico`, verificando una respuesta HTTP 201 y la existencia del identificador.

El riesgo de **inventario inconsistente después del pago** será validado mediante una prueba asíncrona que confirme que el mensaje de actualización de inventario alcance el estado COMPLETED dentro del tiempo establecido.

Para los procesos que puedan quedar en estados intermedios, se verificará que un proceso fallido termine explícitamente en estado FAILED y que nunca permanezca indefinidamente en PENDING.

El flujo completo de pago será validado mediante una prueba E2E denominada `test_flujo_completo_punta_a_punta`, verificando que el usuario llegue correctamente a la pantalla de confirmación.

Finalmente, la degradación bajo carga será evaluada mediante el escenario `onboarding_load.js`, utilizando `k6` y verificando los umbrales establecidos para latencia, errores e idempotencia.